

车辆智能调度 Agent —— 项目答辩稿

配套 PPT： 车辆智能调度Agent-项目答辩.pptx （12 页，与本文档逐页对应）
建议时长： 12 分钟讲解 + 8 分钟问答
语速参考： 230 字/分钟。加粗处是**重音**，|| 表示停顿（换气 + 让听众消化）。
本文档与既有《演讲讲稿-3分钟.md》的关系： 那份是课程演讲用的 3 分钟精简版；
本文档是为答辩扩写的完整版，保留了原稿最核心的三个问题叙事，并补齐技术细节与预设问答。

一、时间轴（贴在手边）

页码	时间	段落	任务
01	0:00-0:20	开场	一句话说清干什么用的
02	0:20-0:50	项目一句话	三个数字立住规模、产出、可信度
03	0:50-1:30	需求与约束	8 条约束 + 为什么人工排不动了
04	1:30-2:10	技术路线	三阶段求解 + 硬约束不交给 LLM
05	2:10-2:55	三个问题	全场重点 ，三个真实故事
06	2:55-3:25	调度全流程	幂等下发 + 异常重排上限
07	3:25-3:50	规则与 AI 分工	边界情况交给规则
08	3:50-4:20	验证结果	279 项可重复通过
09	4:20-4:45	已知限制	主动交代，不等老师问
10	4:45-5:05	系统架构	四层，规则集中在服务层
11	5:05-6:05	关键取舍	预设问答四条
12	6:05-6:20	总结	落点 + 致谢
—	6:20-12:00	现场演示	见第五节动线
—	12:00-20:00	问答	见第六节

如果只给 8 分钟：砍掉第 10、11 页，其余照讲（第 5 页不能砍）。

二、逐字稿

01 开场 (0:00-0:20)

【01】 各位老师好。我汇报的是**车辆智能调度 Agent**。

【01】 一句话说清它是干什么的：每天门店的货量报上来以后，它算出一张排车表 —— || **哪台车、跑几趟、上午下午分别送哪些门店、每趟装多少货**，然后交给调度员确认、下发。

【01】 这个项目的定位是：**把调度员的经验固化成规则，把「排车」从手工劳动变成「选方案」**
—— **人做决策，机器做计算。**

02 项目一句话 (0:20-0:50)

【02】 先用三个数字说清这个项目。

【02】 **第一，规模。**车队 **40 台** —— 四米二 28 台、大包 3 台、小包 9 台；服务 **16 个门店**、5 条线路、19 条门店线路映射。门店和线路是**多对多**关系。

【02】 **第二，产出。**系统一次给出 **4 套方案**：A 四米二优先、B 成本最低、C 大包小包保障、D 装载率均衡，每套带评分和推荐理由，横向比选。

【02】 **第三，可信度。**后端 173 项、前端 106 项，共 **279 项自检全部通过**，而且**可以重复执行**
—— 跑第二遍不会失败。

【02】 **调度员仍然做决策** —— 系统负责算，他负责选。

03 需求与约束 (0:50-1:30)

【03】 场景是门店配送。需求文档归纳了 **8 条业务约束**，最要命的是这四条：

【03】 第一，**装够最低装载量才能发车** —— 四米二最低 630、最高 800；

【03】 第二，**上午门店只能上午送，下午门店只能下午送**；

【03】 第三，车辆有**地形限制** —— 门店分普通、中控、严控，车辆分全能去、大小包能去、小包能去；

【03】 第四，门店和线路是**多对多**，还有门店卡在几条线路的交界处。

【03】 另外，货量不够时优先保障大包、小包的出车趟次；车型上优先用四米二；也不保障每天 28、3、9 台满勤，可以动态调节。

【03】 为什么要做？因为**这件事人工已经排不动了**。

【03】 每一条约束都在互相顶：你想装满一辆四米二，就得把两三个门店拼到一趟 —— 可拼到一起之后，**可能地形不让过、线路对不上，或者其中一个是下午店**。||

【03】 这里有一组数字最能说明问题：**最高单店日货量是 2290，而四米二单车上限只有 800**。一趟车物理上就装不完。

【03】 调度员靠经验排一次要几个小时；排完还说不清为什么这么排，换个人排，结果完全不一样。

04 技术路线 (1:30-2:10)

【04】 实现上是 Vue 3 前端加 FastAPI 后端，28 张表。核心是一个**三阶段混合求解器**。

【04】 **阶段一，启发式。**毫秒级出一个可行解 —— 按车型优先级排序，再用 best-fit 贪心装车。它的作用是**兜底**：保证任何输入都有结果，不会因为求解超时交不出方案。

【04】 **阶段二，CP-SAT。**用 OR-Tools 按每套方案各自的目标权重做精确优化，设了超时上限，超时自动回退到阶段一的结果。

【04】 **阶段三，多方案。**输出 A/B/C/D 四套，每套带评分，交给调度员比选。

【04】 这里有一条**设计原则**必须强调：**硬约束绝不交给 LLM**。LLM 只做方案解释和自然语言生成 —— 就算它算错了，也**不影响方案的可行性**。可行性由代码保证，不由模型保证。

05 三个真正卡过我的问题 (2:10-2:55) ★ 全场重点

【05】 接下来讲三个真正卡过我的问题。这段是我最想讲的。

问题一：这是需求的空白，不是代码的 bug

【05】 我最初严格按文档实现 —— 一个门店的货量由同一趟车一次送完。

【05】 结果实测覆盖率只有 25%，方案基本不能用。

【05】 我去翻数据才发现：有个门店日货量是 2290，而四米二单车上限只有 800 —— 一趟车物理上就装不下。

【05】 需求文档没有禁止拆单，但也没说要拆。这是文档的空白。我跟业务口径核对之后，改成允许一个门店的货量拆到多个趟次，覆盖率从 25% 提到 99.8%。

【05】 这件事让我记住一句话：需求文档没写的东西，不等于业务不需要。||

问题二：不能让求解器自己验自己

【05】 求解器内部已经判过那 7 条硬约束了。但我没有拿它的输出去当校验结果。

【05】 为什么？因为万一求解器本身有 bug，它错的同时也会「认为」自己对 —— 自己验自己等于没验。

【05】 所以我另外写了一个独立的 `validate_solution()`，用完全独立的一套代码把 7 条硬约束重新复核一遍。

【05】 求解器的 bug，绝不能被它的校验逻辑掩盖。||

问题三：编不出来的数据，我不编

【05】 需求里要求监控 API 的 QPS、P95 延迟、Redis 命中率，还要跟 TMS、WMS、OMS 打通。这些都需要额外的中间件和外部系统，本项目没有接入。

【05】 我的处理是 —— 页面上如实标注「未接入」「未连通」，同时说明本地已经落地了哪些链路。

【05】 因为编造的监控数据，比没有监控更危险。||

【05】 这三件事合起来说明一个态度：需求文档只是起点，真正的业务口径得自己去核；验证要独立于实现；交不出的部分如实说明，拿数字糊过去只会埋雷。

06 调度全流程（2:55-3:25）

【06】 这是从货量数据到派车指令的完整链路，六个节点：

【06】 数据感知 → 约束解析 → 规则校验 → 方案生成 → 人工确认 → 下发。

【06】 其中两个工程细节我想专门说。

【06】 第一个是下发幂等。`dispatch_record` 上对「任务 + 方案 + 趟次」建了唯一约束。重复点「下发」会返回「新增 0、跳过 47」，不会产生重复任务。演示的时候我会故意连点两次，就是这个效果。

【06】 第二个是异常重排。车辆故障、门店加减货可以上报异常触发重排。已经执行过的趟次锁死不动；重排次数有上限，默认 3 次，连点到上限会明确拒绝，避免死循环。

【06】 这一页的关键取舍是：机器负责算，人负责决定。方案不自动执行，必须经调度员确认才下发。

07 规则与 AI 的分工（3:25-3:50）

【07】 这一页只讲一个判断：边界情况交给规则，不交给 AI。

【07】 系统分三层。**规则层**管 8 条业务约束、装载量门槛、时段归属、地形通行矩阵、线路映射；**求解层**负责求可行解和优化目标；**LLM 层**只生成方案解释、调度报告和自然语言摘要，**不参与任何约束判断**。

【07】 举三个规则层实际拦下的例子：

【07】 第一，**趟次拆分不自洽**——上午趟次加下午趟次不等于每日趟次，直接拒绝，不允许存进配置；

【07】 第二，**地形通行不允许**——严控地形禁止小包车型进入，矩阵在保存时就校验；

【07】 第三，**整数参数写入非数字**——求解超时等参数要求整数，非数字直接报错不落库。

【07】 一句话总结这一页：**如果一个约束算错了会导致方案不可行，它就必须写在代码里，而不是交给模型判断。**

08 验证结果 (3:50-4:20)

【08】 验证要靠**可重复执行的脚本**说话。

【08】 **后端 173 项**，分 6 个脚本：`check_api.py` 42 项覆盖认证、权限并集、停用角色失效、CRUD、删除保护、审计只追加；`check_solver.py` 17 项专门复核 7 条硬约束；`check_scheduling.py` 35 项覆盖确认、下发幂等、重排上限、调度报告；另外还有货量、报表口径、规则版本与监控。

【08】 **前端 106 项**，分 5 个脚本，用 **Playwright 驱动本机真实 Chrome**，同时收集浏览器控制台报错——正常运行应该是 **0 条**。

【08】 合计 **279 项，全部可重复通过**。所谓可重复，是指**连续跑两遍结果一致**，这本身就证明了操作的幂等性。

【08】 补充一句：前端还有 40 个 `.vue` 组件的模板编译检查，以及一个静态检查脚本，专门查「禁用状态的字段有没有正确回填」这种容易漏的细节。

09 已知限制 (4:20-4:45)

【09】 这一页我要主动交代。

【09】 需求里的 **9 大模块，6 个完整落地**：系统管理、基础数据与规则配置中心、车辆分配、智能调度 Agent、报表与看板、监控预警。

【09】 **3 个部分完成**，我如实标注：

【09】 第六，**移动端 / 司机端**——需求里定位为独立端，后端接口已就绪，但没有建独立前端工程；

【09】 第七，**接口集成与数据交换**——TMS/OMS/WMS/ERP/地图/GPS **全部未真实连通**，页面标 `not_connected`，并说明本地已落地哪些链路；

【09】 第九，**数据与算法平台**——求解器和规则中心已实现，**向量库与案例沉淀是规划项**。

【09】 另外四项也主动说明：附件管理只登记元信息不做真实落盘；API QPS 和 P95 延迟需要 Prometheus 等外部组件，未引入所以**页面不展示这些数字**；成本是**相对系数**（四米二 1.0、大包 0.85、小包 0.5），只用于方案间横向比较，不是真实金额；路径规划用「一趟最多 8 个门店」简化，**未接入真实地图**。

【09】 我宁愿少写几个功能，也不要放一个看起来能跑、实际上经不起查的东西上去。

10 系统架构 (4:45-5:05)

【10】 架构分四层，**规则全部集中在服务层**，接口层不重复判断，避免规则散落导致不一致。

【10】 **表现层**是 Vue 3，28 个页面视图。菜单和按钮按服务端返回的权限动态裁剪——但要强调，**前端裁剪只是体验优化，最终防线在后端**。

【10】 接口层是 FastAPI，用统一的 `require_permission()` 鉴权。权限模型这里有个容易错的地方：**有效权限 = 所有启用角色权限的并集，而且权限点自身也须启用**；用户没有任何角色时权限是空集，所有受保护接口返回 403。

【10】 服务层是核心，求解、权限、审计、规则、报表都在这层。

【10】 数据层是 MySQL 8.0，28 张表。审计日志用四层保证只追加：代码层只有 INSERT、接口层只有 GET、模型层操作人可空、数据层冗余操作人快照。

【10】 关于数据库选型：文档推荐 PostgreSQL，我用了 MySQL —— 原因写在页面底部，**代码里没有使用任何 MySQL 方言，换 PG 只需改一行配置**。

11 关键取舍与预设问答 (5:05-6:05)

【11】 最后一页我把可能被问到的问题和答案直接列出来。每个偏离文档的决定，都有可核对的理由。

【11】 问：为什么用 MySQL，文档写的是 PostgreSQL？

【11】 答：本机没有 PG，Docker 守护进程也没起。但**代码里没有使用任何 MySQL 方言**，换 PG 只需要改 `config.py` 的 `drivername` 一行，模型不用动。

【11】 问：LLM 到底用在哪？

【11】 答：只做方案解释和报告生成。**硬约束绝不交给 LLM**，算错也不影响可行性。LLM 不可用时会降级为规则引擎输出。

【11】 问：允许拆单是不是在回避难点？

【11】 答：**恰恰相反**。拆单把问题从「门店 → 车辆」变成「货量单元 → 趟次槽位」的分配问题，**建模更难**。2290 超过 800 是物理约束，不拆就是 25% 覆盖率。

【11】 问：测试怎么做的？

【11】 答：后端 6 个脚本 173 项，前端 5 个脚本 106 项，共 279 项，全部可重复执行；前端用 Playwright 驱动真实浏览器并收集控制台报错。

【11】 原则是：**偏离文档可以，但每一个偏离都要能说清原因，并且能当场核对**。

12 总结 (6:05-6:20)

【12】 总结一下。

【12】 目前 **9 大模块里 6 个完整落地**，后端 173 项、前端 106 项自检全部可重复通过，货量覆盖率 99.8%。

【12】 这个项目做的是**可执行、可解释、可追溯**的调度规则：

【12】 **可执行** —— 方案能直接下发派车；

【12】 **可解释** —— 每套方案带评分与推荐理由；

【12】 **可追溯** —— 下发记录与审计日志只追加。

【12】 一句话：**这个项目把调度经验做成了可执行、可解释、可追溯的规则**。

【12】 感谢各位老师，请提问。

三、现场演示动线 (6:20 起，约 3-5 分钟)

只走一条主线，**不要翻页乱点**。提前把浏览器开好、登录好。

步	操作	要念的一句话
1	业务基础数据 → 门店配送需求 → 点「生成演示货量」	「16 个门店，日货量约 19920。」
2	智能调度 Agent → 调度任务 → 点「开始调度」	「等几秒 —— 启发式先出解，CP-SAT 再优化。」
3	多方案比选	「A 方案 29 趟、15 车、四米二使用率 86.2%、货量满足 100%。」
4	人工确认 → 确认 → 下发 → 再点一次下发	「看， 新增 0、跳过 47 —— 这就是幂等。」
5	异常重排 → 上报异常 → 连点到上限	「到上限会明确拒绝， 不会死循环 。」
6	调度规则配置 → 规则版本治理	「改个参数 → 发布新版本 → 看变更摘要 → 回滚。」

【12】 如果时间紧，只演第 4 步的「重复下发」—— 这一下最能证明你想过工程问题。

【12】 第 6 步的版本回滚是第二优先，能证明规则可治理。

四、演示前 5 分钟检查清单

- ☐ 后端 `python run.py` 已起 (8000) , 前端 `pnpm run dev` 已起 (5175)
- ☐ 浏览器已登录 `admin` , 停在第 1 步页面
- ☐ 确认货量数据已生成 (否则第 2 步没有输入)
- ☐ 语速: **第一段刻意放慢, 开头 15 秒决定老师听不听后面**
- ☐ 数字念准: **28/3/9、40 台、16 门店、630/800、2290、25%、99.8%、279 项、173/106**
- ☐ 讲第 5 页「三个问题」时**不要道歉、不要心虚** —— 这是加分项
- ☐ 两句金句说完**停一秒**:
 - 「**人做决策, 机器做计算。**」
 - 「**编造的监控数据, 比没有监控更危险。**」
- ☐ 超时优先砍第 10、11 页, **第 3、5、9 页不能砍**

五、预设问答 (20 分钟问答用)

按「先给结论, 再给依据」回答。每题控制在 40 秒内。

可能的提问	回答要点
为什么用 MySQL，文档写的是 PostgreSQL？	本机无 PG 且 Docker 未起。 代码未使用任何 MySQL 方言 ，换 PG 只改 <code>config.py</code> 一行 <code>drivername</code> 。
LLM 到底用在哪儿？	LangGraph 节点已就位。 硬约束绝不交给 LLM —— 只做方案解释和报告生成，算错也不影响可行性。
允许拆单是不是在回避难点？	恰恰相反 。拆单把问题变成「货量单元 → 趟次槽位」的分配问题，建模更难。2290 超过 800 是物理约束，不拆就是 25% 覆盖率。
为什么 D 方案是 99.8%，不是 100%？	覆盖率口径是 按货量满足比例 算的。D 方案只有 25 趟、13 台车，尾部凑不满最低装载量的零头会被放弃；A 和 C 是 100%。这个口径差异在报表和自检脚本里都写明了。
测试怎么做的？	后端 6 个脚本 173 项，前端 5 个脚本 106 项，共 279 项，全部可重复执行 （跑第二遍不会失败）。前端用 Playwright 驱动本机 Chrome，同时收集控制台报错，正常应为 0 条。
哪些没做？	移动端/司机端未实现（需求里定位为独立端）；TMS/OMS/WMS/ERP 全部未真实连通，页面标 <code>not_connected</code> ；向量库和案例沉淀是规划项。 共 9 大模块，6 个完整、3 个部分完成 。
成本是怎么算的？	四米二 1.0 / 大包 0.85 / 小包 0.5，是 相对系数 ，只用于方案之间横向比较，不是真实金额 —— 页面上有标注。
求解器会不会算不出来？	不会。 阶段一是启发式兜底 ，毫秒级必出可行解；CP-SAT 超时会回退到阶段一的结果。
四套方案的权重怎么定？	权重在 规则配置中心 里可配、可版本化。所以调权重不用改代码，发布新版本即可，并且能回滚。
权限模型有什么坑？	三个易错点：① 角色被停用 → 该角色贡献的权限 整块失效 （不是取最严）；② 权限点被停用 → 即使角色仍引用它也不生效；③ 用户无任何角色 → 权限空集，所有受保护接口 403。
审计日志能改吗？	不能。四层保证只追加：代码层只有 <code>services/audit.py</code> 写日志且只 INSERT；接口层 <code>/api/audit-logs</code> 只有 GET，其余方法 405 ；模型层操作人可空；数据层冗余操作人快照。
重排会不会死循环？	不会。重排次数有上限（默认 3）， 连点到上限会明确拒绝 。已执行过的趟次锁死不动。
这个项目最难的地方是什么？	约束之间的耦合 ：装载量、时段、地形、线路四条约束互相顶，任何一条单独看都简单，合起来就变成组合爆炸。
如果让你重做，会改什么？	先做 数据勘察 再写代码 —— 2290 这个数字如果一开始就发现，能省掉一轮返工。

六、被问到不会的问题怎么办

【12】 「这个问题我目前没有验证过，不能凭印象回答。我记下来，答辩结束后查清再回复您。」

不要现场猜。 这个项目的立场就是「编不出来的数据我不编」——现场编一个答案，会把前面 279 项可重复验证的可信度一起赔进去。